

Learn2Slither

Apprentissage par renforcement

Résumé : Un serpent qui apprend à se comporter dans un environnement par essais et erreurs.

Version : 1.00

Sommaire

I. Avant-propos

II. Introduction

III. Règles générales

IV. Partie obligatoire

IV.1 Partie 1 : Environnement / Plateau

IV.2 Partie 2 : État

IV.3 Partie 3 : Action

IV.4 Partie 4 : Récompenses

IV.5 Partie 5 : Q-learning

IV.6 Partie 6 : Structure technique

V. Rendu et évaluation par les pairs

VI. Partie bonus

Chapitre I — Avant-propos

Dans les mystérieux replis de l'espace-temps, où les étoiles fredonnent d'absurdes mélodies et où les planètes organisent des rencontres inattendues, existe un coin particulier de l'univers connu sous le nom de « La Galaxie Serpentine ». Ce secteur interstellaire est réputé pour ses serpents extraterrestres, qui débattent avec de profondes nuances philosophiques tout en s'enroulant gracieusement autour d'astéroïdes dérivants dans le vide.

Les serpents de la Galaxie Serpentine, dont les écailles irisées sont imprégnées de questions existentielles, se rassemblent en spirales cosmiques pour discuter des grands mystères de la vie, de l'univers et de tout ce qui se trouve entre deux boucles de serpent. Leur langage sifflant est une étrange symphonie de pensées ophidiennes, évoquant des concepts et des idées tortueux qui semblent aussi insaisissables qu'un astéroïde glissant entre les doigts.

Certains affirment que ces serpents de l'espace sont les gardiens de secrets cosmiques, tandis que d'autres murmurent qu'ils sont les sages érudits d'une dimension serpentine parallèle. Quoi qu'il en soit, la Galaxie Serpentine demeure un lieu mystérieux où les serpents, avec leur sagesse sinueuse, continuent de débattre des grandes questions de l'existence, enroulant leurs pensées autour de chaque énigme cosmique avec une élégance qui pourrait défier toute logique de l'univers.

Douglas Adams, Le Restaurant au bout de l'Univers

Chapitre II — Introduction

Ce projet est un projet d'intelligence artificielle portant sur l'apprentissage par renforcement.

L'apprentissage par renforcement (RL) est à l'avant-garde de l'intelligence artificielle. Il s'agit d'un paradigme dans lequel des agents intelligents apprennent des stratégies de prise de décision optimales en interagissant avec leur environnement. Contrairement aux approches traditionnelles de programmation, le RL permet aux machines d'apprendre par essais et erreurs, en améliorant continuellement leur comportement grâce aux retours qu'elles reçoivent sous forme de récompenses ou de punitions.

Dans ce projet, vous allez créer un serpent se déplaçant sur un plateau et contrôlé par un agent intelligent. Le plateau et l'agent suivent des règles et contraintes précises, détaillées ci-dessous. Nous vous encourageons à lire attentivement l'intégralité du sujet avant de commencer tout développement ; il serait regrettable de créer la partie plateau sans tenir compte des règles qui s'appliquent à l'agent.

Évidemment, l'agent contrôlant le serpent devra mettre en œuvre l'apprentissage par renforcement pour prendre ses décisions. Chaque décision prise par l'agent entraînera un retour positif ou négatif de la part du plateau. L'agent s'adaptera alors afin de maximiser les retours positifs (ou récompenses) au fil du temps. Ce processus d'apprentissage est comparable à la manière dont les humains et les animaux apprennent par l'expérience, ce qui fait du RL un cadre puissant pour résoudre des problèmes complexes dans de nombreux domaines.

Chapitre III — Règles générales

Vous pouvez utiliser directement un ordinateur dans votre cluster préféré ou une machine virtuelle.

Concernant la machine virtuelle :

- Vous êtes libre de choisir le système d'exploitation que vous souhaitez.
- Vous installerez sur la machine virtuelle tous les logiciels nécessaires à la réalisation de votre projet.

Concernant un ordinateur du campus :

- Assurez-vous que tous les outils nécessaires sont installés sur l'ordinateur ou que vous pouvez les installer localement sur votre compte.
- Assurez-vous d'avoir suffisamment d'espace dans votre session locale pour installer tout ce dont vous avez besoin pour les différents modules (utilisez goinfre si votre campus en dispose).
- Tout doit être installé avant les évaluations.

Comme d'habitude, votre programme ne doit pas quitter de manière inattendue (selon le langage utilisé), en dehors des comportements indéfinis. Si cela se produit, votre projet sera considéré comme non fonctionnel et recevra la note de 0 pendant l'évaluation.

Vous devez soumettre votre travail principal (le plateau, l'agent et vos modèles entraînés) dans le dépôt Git qui vous a été attribué. Seul le contenu du dépôt Git sera évalué : un « git clone » du dépôt sera effectué dans un nouveau répertoire vide, où l'environnement d'exécution approprié sera disponible (machine virtuelle, environnement virtuel Python, etc.).

Vous êtes libre de choisir le langage de programmation de votre choix pour ce projet, mais nous recommandons Python car de nombreuses bibliothèques peuvent vous aider. Si vous réalisez ce projet en Python, il doit respecter la norme.

- `pip install flake8`
- `alias norminette_python=flake8`

Amusez-vous avec l'entraînement de votre serpent !

Chapitre IV — Partie obligatoire

IV.1 Partie 1 : Environnement / Plateau

L'environnement du serpent est représenté par le plateau. Les règles suivantes s'appliquent au plateau :

- Taille du plateau : 10 cases sur 10.
- Deux pommes vertes, placées dans des cases aléatoires du plateau.
- Une pomme rouge, placée dans une case aléatoire du plateau.
- Le serpent commence avec une longueur de 3 cases, également placé aléatoirement et de manière contiguë sur le plateau.
- Si le serpent touche un mur : fin de la partie, cette session d'entraînement prend fin.
- Si le serpent entre en collision avec sa propre queue : fin de la partie, cette session d'entraînement prend fin.
- Le serpent mange une pomme verte : sa longueur augmente de 1. Une nouvelle pomme verte apparaît sur le plateau.
- Le serpent mange une pomme rouge : sa longueur diminue de 1. Une nouvelle pomme rouge apparaît sur le plateau.
- Si la longueur du serpent tombe à 0 : fin de la partie, cette session d'entraînement prend fin.

Sessions d'entraînement (ou parties, ou manches) :

Un agent non entraîné échouera rapidement. Une seule session d'entraînement ne peut pas suffire à apprendre : l'agent devra faire face à des centaines ou des milliers de sessions d'entraînement pour améliorer ses compétences. Le programme principal devra proposer un paramètre en ligne de commande permettant de définir le nombre de sessions d'entraînement à exécuter.

Pour garantir le bon fonctionnement du projet, vous devez fournir une interface graphique affichant le plateau et ses éléments au fil du temps (chaque décision de l'agent met à jour le plateau). La vitesse d'affichage doit être configurable et au moins une vitesse lisible par un humain doit exister. Un mode pas à pas doit également être disponible.

Learn2Slither — Apprentissage par renforcement

- Vert : pomme verte
- Rouge : pomme rouge
- Bleu : serpent

Learn2Slither — IV.2 Partie 2 : État

Vision du serpent :

Votre serpent ne peut voir que dans les 4 directions depuis sa tête. La figure représente la sortie terminale que votre programme calculera avant de demander à l'agent où le serpent doit se déplacer. Cette sortie correspond à la représentation du plateau présentée précédemment.

Dans cette figure, chaque lettre a bien entendu une signification précise :

- W = Mur
- H = Tête du serpent
- S = Segment du corps du serpent
- G = Pomme verte
- R = Pomme rouge
- 0 = Espace vide

Vous pouvez uniquement fournir à l'agent les informations visibles par votre serpent. Fournir davantage d'informations entraînera une pénalité de -42.

Learn2Slither — IV.3 Partie 3 : Action

Votre agent ne peut effectuer que 4 actions (HAUT, GAUCHE, BAS, DROITE). Il doit prendre ses décisions uniquement à partir de la vision du serpent (État), depuis la tête et dans les 4 directions. Aucune autre information provenant du plateau ne doit être utilisée par l'agent.

Le plateau ou l'environnement est affiché graphiquement dans une fenêtre dédiée. L'état ou la vision ainsi que l'action effectuée par l'agent sont affichés dans le terminal, conformément à la figure précédente.

Learn2Slither — IV.4 Partie 4 : Récompenses

L'objectif de votre serpent est d'atteindre au moins une longueur de 10 cases et de rester en vie aussi longtemps que possible. L'action choisie par l'agent évoluera au fil du temps afin d'atteindre cet objectif, en fonction des récompenses attribuées par l'environnement (ou le plateau) après chaque action.

C'est à vous de définir les récompenses positives et négatives. Une approche évidente pourrait être :

- Si le serpent mange une pomme rouge : une récompense négative.
- Si le serpent mange une pomme verte : une récompense positive.
- Si le serpent ne mange rien : une récompense négative plus faible.
- Si le jeu se termine (collision avec un mur, collision avec lui-même, longueur nulle) : une récompense négative plus importante.

La récompense de chaque action augmentera ou diminuera la probabilité que l'agent effectue le même choix à l'avenir lorsqu'il se trouvera dans une situation identique.

Learn2Slither — IV.5 Partie 5 : Q-learning

Vous devez implémenter un modèle qui utilise une fonction Q pour évaluer la qualité d'une action dans un état donné. Cette fonction Q peut être implémentée sous forme de valeurs Q dans une table Q ou à l'aide d'un réseau neuronal.

Aucun autre modèle n'est autorisé ; dans le cas contraire, une note de 0 sera attribuée.

Mise à jour de la fonction Q :

L'algorithme Q-learning est conçu pour ajuster la fonction Q (et les valeurs Q associées) en fonction de la récompense reçue après chaque action effectuée.

Vous pouvez entraîner plusieurs modèles en utilisant différentes méthodes de mise à jour de la fonction Q.

Exploration contre exploitation :

Pour découvrir de nouvelles actions qui pourraient être bénéfiques, l'agent doit parfois effectuer des actions aléatoires au lieu de toujours choisir celles qui semblent être les meilleures.

Apprentissage itératif :

L'agent répète ce processus d'interaction avec l'environnement, en prenant des actions, en recevant des récompenses et en mettant à jour la fonction Q de manière itérative. Lorsque la session d'entraînement est terminée, une nouvelle session est démarrée afin de poursuivre l'apprentissage.

Exporter et importer les modèles :

À tout moment, il est possible d'exporter un fichier unique contenant tout ce qui décrit l'état d'apprentissage actuel de l'agent. Ce fichier contiendra principalement les valeurs Q. Il est possible d'importer ce fichier pour restaurer l'apprentissage de l'agent à un niveau donné. Chaque fichier constitue un « modèle » de votre IA. Vous devrez rendre plusieurs modèles dans votre dépôt Git.

Learn2Slither — IV.5 Partie 5 : Q-learning (suite)

Exploitation sans apprentissage :

Un paramètre de configuration de l'agent empêchera le processus d'apprentissage de se produire. L'agent ignorera simplement les récompenses du plateau et la fonction Q ne sera pas mise à jour. Cette option est utile pour évaluer le taux de réussite d'un modèle spécifique sans le modifier.

Pendant la phase d'entraînement de votre agent, il est possible de supprimer l'affichage graphique ainsi que les sorties du terminal afin d'accélérer le processus.

Learn2Slither — IV.6 Partie 6 : Structure technique

Veillez vous assurer que votre programme est modulaire, comme indiqué dans le schéma du sujet, afin de faciliter l'évaluation de chaque partie du projet. Il vous appartient de déterminer la manière la plus efficace pour chaque module de communiquer avec les autres.

Chapitre V — Rendu et évaluation par les pairs

Rendez votre travail (plateau, agent et modèles) via votre dépôt Git, comme d'habitude. Seul le contenu présent dans votre dépôt sera évalué pendant la soutenance.

Vous devez avoir un dossier « models » contenant vos modèles entraînés, enregistrés dans un format de fichier de votre choix (par exemple, .txt). Chaque modèle peut être plus ou moins entraîné, après quelques sessions ou de nombreuses sessions, et avec différentes méthodes de mise à jour de la fonction Q.

Vous devez avoir effectué votre entraînement à l'avance et généré plusieurs modèles ; soyez conscient que cela peut prendre beaucoup de temps.

Exemple de commande en ligne et d'affichage :

```
$> ./snake -sessions 10 -save models/10sess.txt -visual off
```

...

Fin de la partie, longueur maximale = 4, durée maximale = 17

Sauvegarde de l'état d'apprentissage dans models/10sess.txt

Vous devez avoir au moins 3 fichiers de modèles sauvegardés, entraînés respectivement avec 1, 10 et 100 sessions d'entraînement. Cela doit montrer comment votre serpent apprend au fil de plusieurs sessions d'entraînement.

Vous devez pouvoir démarrer une nouvelle session à partir de l'un de vos modèles sauvegardés, en combinaison avec la fonctionnalité « sans apprentissage », afin de vérifier les performances de chaque modèle. Cela sera testé pendant la soutenance.

Exemple :

```
$> ./snake -visual on -load models/100sess.txt -sessions 10 -dontlearn -step-by-step
```

Charger le modèle entraîné depuis models/100sess.txt

...

Fin de la partie, longueur maximale = 7, durée maximale = 32

Votre objectif est d'avoir un serpent d'une longueur de 10 ou plus à la fin de la session, ainsi qu'une durée de vie importante. Vous aurez donc probablement besoin de beaucoup d'entraînement.

Learn2Slither — Exemple

```
$> ./snake -visual on -load models/1000sess.txt
```

```
...
```

Fin de la partie, longueur maximale = 12, durée maximale = 55

Chapitre VI — Partie bonus

Les bonus ne seront pris en compte que si toutes les parties obligatoires sont correctes.

Liste des bonus acceptés :

- Atteindre une longueur plus élevée à la fin d'une session (15, 20, 25, 30, 35).
- Créer un affichage visuellement impressionnant, avec un lobby, un panneau de configuration, des résultats et des statistiques, etc.
- Permettre de modifier la taille du plateau avec des arguments (votre serpent doit pouvoir « jouer » avec le même modèle entraîné quelle que soit la taille du plateau, afin de valider ce bonus).